

Student Enrollment & Timetable System

Object-Oriented Programming (OOP) Final Project Report

Author: Md Fuyad Hassan (ID: 232-134-043)

Affiliation: Dept. of Software Engineering, Metropolitan University

Course: Object Oriented Programming (OOP)

Development Stack: Java GUI (Swing) & SQLite RDBMS via JDBC

Submission Date: August 15, 2026



Abstract

This report details the implementation of an integrated desktop platform designed to manage student registration and academic course scheduling. Built using Java Swing GUI controls and a relational SQLite database connected through JDBC, the application implements a normalized schema designed to resolve a classic many-to-many relationship using a junction timetable entity. In addition, the development incorporates a custom Date validation component, an interactive visual date picker, custom Java enums mapped to user controls, and database cascading operations to maintain relational integrity. The design adheres strictly to standard Object-Oriented Programming principles and enterprise naming conventions.

1. Project Description

The **Student Enrollment and Timetable Management System** is a desktop software application developed to manage academic student enrollments and course registration details. The application redevelops the architectural concepts originally implemented in a Python-based GUI database project into a type-safe, compiled Java Swing environment.

From a database modeling perspective, the relationship between Students and course Units is inherently **many-to-many**: a single student can enroll in multiple units, and an individual unit can contain numerous registered students. To represent this structure in an RDBMS, the **Timetable** entity serves as a **Link Entity** (junction table). Each record in the timetable table links a unique student record with a corresponding course unit code, while scheduling day and time attributes are mapped directly to this linkage.

The software design incorporates several critical components to satisfy OOP guidelines:

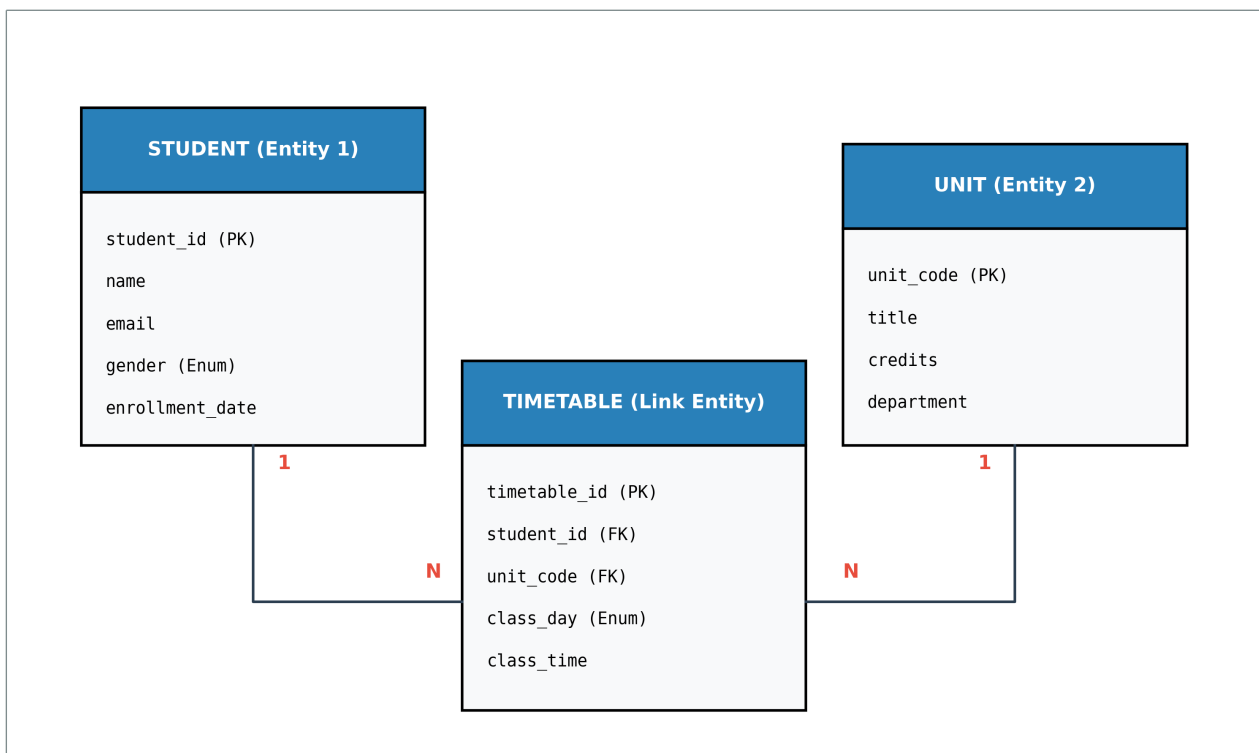
- **Java Swing Desktop GUI:** A multi-tab dashboard (JTabbedPane) that integrates three independent forms for Students, Units, and Schedules. Each form is paired with an interactive JTable representing the live state of the underlying SQLite tables.
- **Custom Date Implementation:** To demonstrate standard calendar logic, a custom MyDate class is written. It encapsulates year, month, and day integer fields, and implements leap-year calculations and monthly day-bound validations to restrict invalid object instantiations.
- **Interactive Date Picker:** To enter date information visually, a custom, grid-based modal dialog (DatePickerDialog) is built in Swing. Users navigate months/years and click days to return

validated `MyDate` objects.

- **Enum Types:** Java `enum` structures (`Gender` and `DayOfWeek`) are implemented for categorical fields, ensuring compile-time safety. In the UI, these are represented using a `ButtonGroup` of radio buttons for gender and a dropdown `JComboBox` for timetable days.
- **Relational Database Cascade:** To prevent orphaned records, foreign key cascade actions are established. If a student or course unit record is deleted, the RDBMS automatically purges all related schedule entries in the timetable link table.

2. Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) defines the schema structure, attributes, and cardinality of the three entities. The central **TIMETABLE** entity functions as the link table resolving the many-to-many relationship:



Cardinality Specifications:

- One **STUDENT** is mapped to zero, one, or many scheduling entries in the **TIMETABLE** table (1 to N relationship). Deleting a student cascades and purges matching schedule entries.
- One **UNIT** is mapped to zero, one, or many scheduling entries in the **TIMETABLE** table (1 to N relationship). Deleting a unit cascades and purges matching schedule entries.

3. Database Relational Schema

The SQLite database structure contains three relational tables. The field-level structures and data constraints are specified in the following tables:

3.1 Table: students

Field Name	Data Type	Constraints	Description
id	INTEGER	PRIMARY KEY	Unique Student ID number
name	TEXT	NOT NULL	Full name of the student
email	TEXT	NOT NULL	Student's email address
gender	TEXT	NOT NULL	Enum value (persisted as MALE, FEMALE, or OTHER)
enrollment_date	TEXT	NOT NULL	Admission date (persisted as string in format YYYY-MM-DD)

3.2 Table: units

Field Name	Data Type	Constraints	Description
unit_code	TEXT	PRIMARY KEY	Alphanumeric course code (e.g., CSE101)
title	TEXT	NOT NULL	Official course title
credits	INTEGER	NOT NULL	Credit hour value (1 to 10)
department	TEXT	NOT NULL	Academic department offering the unit

3.3 Table: timetable (Junction / Link Table)

Field Name	Data Type	Constraints	Description
timetable_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Unique schedule record ID
student_id	INTEGER	FOREIGN KEY -> students(id) ON DELETE CASCADE	Reference link to student
unit_code	TEXT	FOREIGN KEY -> units(unit_code) ON DELETE CASCADE	Reference link to course unit
class_day	TEXT	NOT NULL	Scheduled class day (Enum value: MONDAY, etc.)
class_time	TEXT	NOT NULL	Time range details (e.g., 09:00 AM - 10:30 AM)

4. Object-Oriented Principles & Design Decisions

The project design implements core Object-Oriented Programming (OOP) paradigms:

- **Encapsulation:** Entity attributes (in `Student`, `Unit`, and `Timetable` classes) are marked as `private`. Access is mediated through public getters and setters, ensuring strict state control.

Validation & State Invariant Protection: The custom `MyDate` class protects its date invariant by validating fields in constructors and setters:

```
public MyDate(int day, int month, int year) {
    if (!isValidDate(day, month, year)) {
        throw new IllegalArgumentException("Invalid date");
    }
    // ...
}
```

- **Separation of Concerns:** SQL interactions are segregated in [`DatabaseManager.java`](file:///e:/SWE/2.1%20semester/OOP/Project1/src/dbms/DatabaseManager.java), while business logic remains in the model classes, and user presentation remains in the Swing frame.
- **Code Conventions:** Class names use UpperCamelCase, variables use lowerCamelCase, database tables use snake_case, and SQL keywords are kept in UPPERCASE to maintain strict readability.